

# Introduction to Reinforcement Learning

David Quarel

August 11, 2026

# What is Reinforcement Learning?

- Deep learning is usually presented in the supervised learning setting:
  - ① We have access to labelled training data
  - ② Agent only sees “real world” once it’s already performing well
  - ③ Data is i.i.d between batches
  - ④ No planning required, future predictions don’t depend on past predictions

# What is Reinforcement Learning?

- Deep learning is usually presented in the supervised learning setting:
  - ① We have access to labelled training data
  - ② Agent only sees “real world” once it’s already performing well
  - ③ Data is i.i.d between batches
  - ④ No planning required, future predictions don’t depend on past predictions
- RL is vastly different: Agent takes actions in an interactive environment, receive scalar reward as feedback. This lends itself to several problems:
  - ① **Sparse reward:** Sparse feedback to do good
  - ② **Reward attribution:** Which action mattered?
  - ③ **No ground truth:** Optimal policy unknown
  - ④ **Explore vs. Exploit tradeoff:**
    - ① **Exploration:** Learn how the world works
    - ② **Exploitation:** Exploit best understanding to get reward
  - ⑤ **(often) Online only:** No clear distinction between training and testing.

# How Much Information is the Machine Given during Learning?

## ► “Pure” Reinforcement Learning (**cherry**)

- The machine predicts a scalar reward given once in a while.

► **A few bits for some samples**

## ► Supervised Learning (**icing**)

- The machine predicts a category or a few numbers for each input
- Predicting human-supplied data
- **10→10,000 bits per sample**

## ► Self-Supervised Learning (**cake génoise**)

- The machine predicts any part of its input for any observed part.
- Predicts future frames in videos
- **Millions of bits per sample**



# How Much **(cherry)** is the Machine Given during Learning?

- ▶ “Pure” **(cherry)** Learning **(cherry)**
  - ▶ The machine predicts a scalar reward given once in a while.
  - ▶ Millions of **(cherry)** samples

- ▶ Supervised Learning (apple)
  - ▶ The machine predicts a scalar reward from a few numbers
  - ▶ Millions of samples
  - ▶ 10,000 bits per sample

- ▶ Self-supervised Learning (cake exercise)
  - ▶ The machine predicts a scalar reward from a few numbers
  - ▶ Millions of samples
  - ▶ 10,000 bits per sample



# Agent-Environment Interaction Loop (MDPs)

- MDP is 4-tuple  $(\mathcal{S}, T, \mathcal{A}, R)$ :
  - *states*  $\mathcal{S}$
  - *actions*  $\mathcal{A}$
  - *transition distribution*  $T : \mathcal{S} \times \mathcal{A} \rightarrow \Delta\mathcal{S}$ .  
 $T(s'|s, a) = \Pr(s_{t+1} = s' | s_t = s, a_t = a)$ .
  - *reward function*  $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$

# Agent-Environment Interaction Loop (MDPs)

- MDP is 4-tuple  $(\mathcal{S}, T, \mathcal{A}, R)$ :
  - *states*  $\mathcal{S}$
  - *actions*  $\mathcal{A}$
  - *transition distribution*  $T : \mathcal{S} \times \mathcal{A} \rightarrow \Delta\mathcal{S}$ .  
 $T(s'|s, a) = \Pr(s_{t+1} = s' | s_t = s, a_t = a)$ .
  - *reward function*  $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$
- Agent has *policy*  $\pi : \mathcal{S} \rightarrow \Delta\mathcal{A}$ .  $\pi(a|s) = \Pr(a_t = a | s_t = s)$ .

# Agent-Environment Interaction Loop (MDPs)

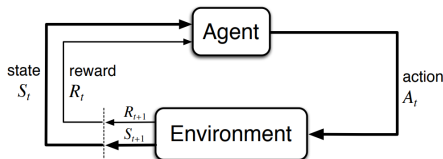
- MDP is 4-tuple  $(\mathcal{S}, T, \mathcal{A}, R)$ :
  - *states*  $\mathcal{S}$
  - *actions*  $\mathcal{A}$
  - *transition distribution*  $T : \mathcal{S} \times \mathcal{A} \rightarrow \Delta\mathcal{S}$ .  
 $T(s'|s, a) = \Pr(s_{t+1} = s' | s_t = s, a_t = a)$ .
  - *reward function*  $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$
- Agent has *policy*  $\pi : \mathcal{S} \rightarrow \Delta\mathcal{A}$ .  $\pi(a|s) = \Pr(a_t = a | s_t = s)$ .
- On timestep  $t$ ,
  - Agent samples  $a_t \sim \pi(\cdot | s_t)$ , gives action  $a_t$  to environment.
  - Environment samples  $s_{t+1} \sim T(\cdot | s_t, a_t)$ , computes  $r_{t+1} = R(s_t, a_t, s_{t+1})$ , and gives *percept*  $(s_{t+1}, r_{t+1})$  to agent.



# Agent-Environment Interaction Loop (MDPs)

- MDP is 4-tuple  $(\mathcal{S}, T, \mathcal{A}, R)$ :
  - states  $\mathcal{S}$
  - actions  $\mathcal{A}$
  - transition distribution  $T : \mathcal{S} \times \mathcal{A} \rightarrow \Delta\mathcal{S}$ .  
 $T(s'|s, a) = \Pr(s_{t+1} = s' | s_t = s, a_t = a)$ .
  - reward function  $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$
- Agent has policy  $\pi : \mathcal{S} \rightarrow \Delta\mathcal{A}$ .  $\pi(a|s) = \Pr(a_t = a | s_t = s)$ .
- On timestep  $t$ ,
  - Agent samples  $a_t \sim \pi(\cdot | s_t)$ , gives action  $a_t$  to environment.
  - Environment samples  $s_{t+1} \sim T(\cdot | s_t, a_t)$ , computes  $r_{t+1} = R(s_t, a_t, s_{t+1})$ , and gives *percept*  $(s_{t+1}, r_{t+1})$  to agent.
- Generates an interaction history, or *trajectory*

$s_0, a_0, r_1, s_1, a_1, r_2, s_2, a_2, r_3, s_3, \dots$



# Objective of the Agent

- At timestep  $t$ , the *return*  $G_t$  is the sum of all future rewards:

$$G_t = r_{t+1} + r_{t+2} + r_{t+3} + \dots$$

# Objective of the Agent

- At timestep  $t$ , the *return*  $G_t$  is the sum of all future rewards:

$$G_t = r_{t+1} + r_{t+2} + r_{t+3} + \dots$$

- **Goal:** Maximise the return.

# Objective of the Agent

- At timestep  $t$ , the *return*  $G_t$  is the sum of all future rewards:

$$G_t = r_{t+1} + r_{t+2} + r_{t+3} + \dots$$

- **Goal:** Maximise the return.
  - For episodic (finite length interaction) environments of maximum duration  $T$ , return  $G_t = r_{t+1} + r_{t+2} + \dots + r_T$  well defined.

# Objective of the Agent

- At timestep  $t$ , the *return*  $G_t$  is the sum of all future rewards:

$$G_t = r_{t+1} + r_{t+2} + r_{t+3} + \dots$$

- **Goal:** Maximise the return.
  - For episodic (finite length interaction) environments of maximum duration  $T$ , return  $G_t = r_{t+1} + r_{t+2} + \dots + r_T$  well defined.
- **Problems:** (for continuing environments)
  - The return may diverge or be undefined

# Objective of the Agent

- At timestep  $t$ , the *return*  $G_t$  is the sum of all future rewards:

$$G_t = r_{t+1} + r_{t+2} + r_{t+3} + \dots$$

- **Goal:** Maximise the return.
  - For episodic (finite length interaction) environments of maximum duration  $T$ , return  $G_t = r_{t+1} + r_{t+2} + \dots + r_T$  well defined.
- **Problems:** (for continuing environments)
  - The return may diverge or be undefined
  - The agent might be lazy

# Objective of the Agent

- At timestep  $t$ , the *return*  $G_t$  is the sum of all future rewards:

$$G_t = r_{t+1} + r_{t+2} + r_{t+3} + \dots$$

- **Goal:** Maximise the return.
  - For episodic (finite length interaction) environments of maximum duration  $T$ , return  $G_t = r_{t+1} + r_{t+2} + \dots + r_T$  well defined.
- **Problems:** (for continuing environments)
  - The return may diverge or be undefined
  - The agent might be lazy
  - The environment is stochastic, and the rewards are often up to chance. How to trade-off unlikely big rewards with likely small rewards?

# Objective of the Agent

- At timestep  $t$ , the *return*  $G_t$  is the sum of all future rewards:

$$G_t = r_{t+1} + r_{t+2} + r_{t+3} + \dots$$

- **Goal:** Maximise the return.
  - For episodic (finite length interaction) environments of maximum duration  $T$ , return  $G_t = r_{t+1} + r_{t+2} + \dots + r_T$  well defined.
- **Problems:** (for continuing environments)
  - The return may diverge or be undefined
  - The agent might be lazy
  - The environment is stochastic, and the rewards are often up to chance. How to trade-off unlikely big rewards with likely small rewards?
  - May desire rewards now to be more valuable than rewards later: \$100 now? Or \$110 in a year?



# Objective of the Agent

- At timestep  $t$ , the *return*  $G_t$  is the sum of all future rewards:

$$G_t = r_{t+1} + r_{t+2} + r_{t+3} + \dots$$

- Goal:** Maximise the return.

- For episodic (finite length interaction) environments of maximum duration  $T$ , return  $G_t = r_{t+1} + r_{t+2} + \dots + r_T$  well defined.

- Problems:** (for continuing environments)

- The return may diverge or be undefined
- The agent might be lazy
- The environment is stochastic, and the rewards are often up to chance. How to trade-off unlikely big rewards with likely small rewards?
- May desire rewards now to be more valuable than rewards later: \$100 now? Or \$110 in a year?

- Solutions:**

- Add a discount factor  $\gamma \in [0, 1)$  so rewards more imminent are worth more, and the return is always well defined.

$$G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots$$

- Want agent to choose actions to maximise the *expected* return.

# Core Assumptions

These kind of environments are called *Markov Decision Processes* (MDPs), and have the following “nice” properties

# Core Assumptions

These kind of environments are called *Markov Decision Processes* (MDPs), and have the following “nice” properties

- 1 **Stationary:** The environmental distribution  $T$  is fixed and does not change over time

# Core Assumptions

These kind of environments are called *Markov Decision Processes* (MDPs), and have the following “nice” properties

- 1 **Stationary:** The environmental distribution  $T$  is fixed and does not change over time
- 2 **Markovian:** The behaviour of the environment depend only on the current state  $s_t$  and action  $a_t$ .

# Core Assumptions

These kind of environments are called *Markov Decision Processes* (MDPs), and have the following “nice” properties

- ➊ **Stationary:** The environmental distribution  $T$  is fixed and does not change over time
- ➋ **Markovian:** The behaviour of the environment depend only on the current state  $s_t$  and action  $a_t$ .
- ➌ **Fully Observable:** The state is a full description of the world

# Core Assumptions

These kind of environments are called *Markov Decision Processes* (MDPs), and have the following “nice” properties

- ❶ **Stationary:** The environmental distribution  $T$  is fixed and does not change over time
- ❷ **Markovian:** The behaviour of the environment depend only on the current state  $s_t$  and action  $a_t$ .
- ❸ **Fully Observable:** The state is a full description of the world
- ❹ **Reward Hypothesis:**

*“That all of what we mean by goals and purposes can be well thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (called reward).” -Rich Sutton*

# Value Function

- Want to define the “goodness” (*value*) of a state, so the agent can take actions to move towards “good” states, and away from “bad” states.

# Value Function

- Want to define the “goodness” (*value*) of a state, so the agent can take actions to move towards “good” states, and away from “bad” states.
- The value of a state depends also on the current policy  $\pi$  chosen by the agent.



# Value Function

- Want to define the “goodness” (*value*) of a state, so the agent can take actions to move towards “good” states, and away from “bad” states.
- The value of a state depends also on the current policy  $\pi$  chosen by the agent.

## Value Function

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | s_t = s]$$

(Expectation is also with respect to the transitional distribution  $T$ .)

# Bellman Equation

We note that since

$$\begin{aligned} G_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \\ &= r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) \\ &= r_{t+1} + \gamma G_{t+1} \end{aligned}$$

# Bellman Equation

We note that since

$$\begin{aligned} G_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \\ &= r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) \\ &= r_{t+1} + \gamma G_{t+1} \end{aligned}$$

we can then define the value function recursively,

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t \mid s_t = s]$$

# Bellman Equation

We note that since

$$\begin{aligned} G_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \\ &= r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) \\ &= r_{t+1} + \gamma G_{t+1} \end{aligned}$$

we can then define the value function recursively,

$$\begin{aligned} V_\pi(s) &= \mathbb{E}_\pi[G_t \mid s_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1} \mid s_t = s] \end{aligned}$$

# Bellman Equation

We note that since

$$\begin{aligned} G_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \\ &= r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) \\ &= r_{t+1} + \gamma G_{t+1} \end{aligned}$$

we can then define the value function recursively,

$$\begin{aligned} V_\pi(s) &= \mathbb{E}_\pi[G_t \mid s_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1} \mid s_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} \mid s_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} \mid s_t = s] \end{aligned}$$

# Bellman Equation

We note that since

$$\begin{aligned}G_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \\&= r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) \\&= r_{t+1} + \gamma G_{t+1}\end{aligned}$$

we can then define the value function recursively,

$$\begin{aligned}V_\pi(s) &= \mathbb{E}_\pi[G_t \mid s_t = s] \\&= \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1} \mid s_t = s] \\&= \mathbb{E}_\pi[r_{t+1} \mid s_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} \mid s_t = s] \\&= \sum_a \pi(a|s) \sum_{s'} T(s'|s, a) R(s, a, s') \\&\quad + \gamma \sum_a \pi(a|s) \sum_{s'} T(s'|s, a) \mathbb{E}_\pi[G_{t+1} \mid s_{t+1} = s']\end{aligned}$$

# Bellman Equation

We note that since

$$\begin{aligned} G_t &= r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots \\ &= r_{t+1} + \gamma(r_{t+2} + \gamma r_{t+3} + \dots) \\ &= r_{t+1} + \gamma G_{t+1} \end{aligned}$$

we can then define the value function recursively,

$$\begin{aligned} V_\pi(s) &= \mathbb{E}_\pi[G_t \mid s_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} + \gamma G_{t+1} \mid s_t = s] \\ &= \mathbb{E}_\pi[r_{t+1} \mid s_t = s] + \gamma \mathbb{E}_\pi[G_{t+1} \mid s_t = s] \\ &= \sum_a \pi(a|s) \sum_{s'} T(s'|s, a) R(s, a, s') \\ &\quad + \gamma \sum_a \pi(a|s) \sum_{s'} T(s'|s, a) \mathbb{E}_\pi[G_{t+1} \mid s_{t+1} = s'] \\ &= \sum_a \pi(a|s) \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_\pi(s')) \end{aligned}$$

This gives the **Bellman equation**

## Bellman Equation

$$V_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$



This gives the **Bellman equation**

## Bellman Equation

$$V_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

- Equation is linear in  $V_{\pi}(\cdot)$ , giving a set of **linear** simultaneous equations.

This gives the **Bellman equation**

## Bellman Equation

$$V_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

- Equation is linear in  $V_{\pi}(\cdot)$ , giving a set of **linear** simultaneous equations.
- Given policy  $\pi$ , can now easily solve for  $V_{\pi}(s_1), V_{\pi}(s_2), \dots$

This gives the **Bellman equation**

## Bellman Equation

$$V_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

- Equation is linear in  $V_{\pi}(\cdot)$ , giving a set of **linear** simultaneous equations.
- Given policy  $\pi$ , can now easily solve for  $V_{\pi}(s_1), V_{\pi}(s_2), \dots$
- Computing  $V_{\pi}$  from  $\pi$  is called **policy evaluation**, denote  $\pi \xrightarrow{E} V_{\pi}$ .

## Bellman Equation (stochastic policy)

$$V_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

Assuming policy  $\pi : S \rightarrow A$  is deterministic

## Bellman Equation (deterministic policy)

$$V_{\pi}(s) = \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

where  $a = \pi(s)$ .

## Bellman Equation (stochastic policy)

$$V_{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \sum_{s' \in \mathcal{S}} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

Assuming policy  $\pi : S \rightarrow A$  is deterministic

## Bellman Equation (deterministic policy)

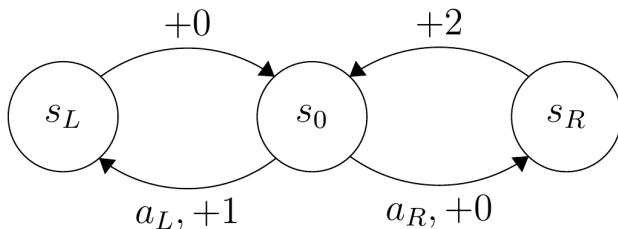
$$V_{\pi}(s) = \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

where  $a = \pi(s)$ .

Only need to sum over all states to find equation for  $V_{\pi}(s)$  in terms of the value of all states  $\{V_{\pi}(s')\}_{s' \in \mathcal{S}}$ .

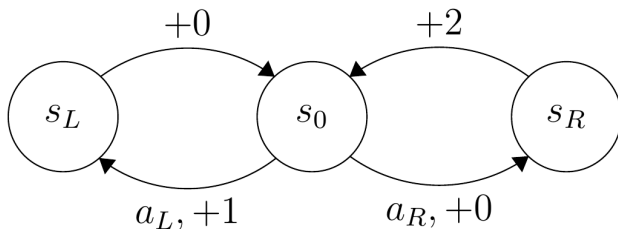
# Example Environment

- States  $\mathcal{S} = s_0, s_L, s_R$ , actions  $\mathcal{A} = \{a_L, a_R\}$ , possible rewards  $\mathcal{R} = \{0, 1, 2\}$ .
- Each transition indicates if an action is taken, the reward returned and which state to transition to
- What is the best action from state  $s_0$ ?



# Example Environment

- States  $\mathcal{S} = s_0, s_L, s_R$ , actions  $\mathcal{A} = \{a_L, a_R\}$ , possible rewards  $\mathcal{R} = \{0, 1, 2\}$ .
- Each transition indicates if an action is taken, the reward returned and which state to transition to
- What is the best action from state  $s_0$ ?



**Exercise:** Depends on the choice of  $\gamma$ ! Verify that

$$V_{\pi_L}(s_0) = \frac{1}{1 - \gamma^2} \quad V_{\pi_R}(s_0) = \frac{2\gamma}{1 - \gamma^2}$$

so breakeven is at  $\gamma = \frac{1}{2}$ .

# Optimal Bellman

- Policy  $\pi_1$  is **better** than  $\pi_2$  ( $\pi_1 \geq \pi_2$ ) if  $\forall s. V_{\pi_1}(s) \geq V_{\pi_2}(s)$ . A policy is **optimal** if it is better than all other policies.



# Optimal Bellman

- Policy  $\pi_1$  is **better** than  $\pi_2$  ( $\pi_1 \geq \pi_2$ ) if  $\forall s. V_{\pi_1}(s) \geq V_{\pi_2}(s)$ . A policy is **optimal** if it is better than all other policies.
- **Theorem:** An optimal policy  $\pi^*$  always exists (may not be unique). Define optimal value function as

$$V_*(s) := \max_{\pi} V_{\pi}(s) \equiv V_{\pi^*}(s)$$

# Optimal Bellman

- Policy  $\pi_1$  is **better** than  $\pi_2$  ( $\pi_1 \geq \pi_2$ ) if  $\forall s. V_{\pi_1}(s) \geq V_{\pi_2}(s)$ . A policy is **optimal** if it is better than all other policies.
- **Theorem:** An optimal policy  $\pi^*$  always exists (may not be unique). Define optimal value function as

$$V_*(s) := \max_{\pi} V_{\pi}(s) \equiv V_{\pi^*}(s)$$

## Optimal Bellman Equation

$$V_*(s) = \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_*(s'))$$

Gives a set of **non-linear** simultaneous equations with variables  $V_*(s_1), V_*(s_2), \dots$

**Problem:** No clear way to solve for  $V_*(\cdot)$

Can't just compute  $V_{\pi}$  using policy evaluation for all  $\pi$ , as there are  $|\mathcal{A}|^{|\mathcal{S}|}$  many to choose from.

# Policy Improvement

- Obviously we have that

$$V_*(s) = \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_*(s')) \quad (1)$$

$$\geq \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_\pi(s')) \quad (2)$$

$$\geq \sum_{s'} T(s'|s, \pi(s)) (R(s, \pi(s), s') + \gamma V_\pi(s')) \quad (3)$$

$$= V_\pi(s) \quad (4)$$

# Policy Improvement

- Obviously we have that

$$V_*(s) = \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_*(s')) \quad (1)$$

$$\geq \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_\pi(s')) \quad (2)$$

$$\geq \sum_{s'} T(s'|s, \pi(s)) (R(s, \pi(s), s') + \gamma V_\pi(s')) \quad (3)$$

$$= V_\pi(s) \quad (4)$$

- Given the value of a policy  $V_\pi$ , can feed it through the optimal Bellman equation (2) to get a better policy  $\pi'$ . Denote  $V_\pi \xrightarrow{I} \pi'$ .

# Policy Improvement

- Obviously we have that

$$V_*(s) = \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_*(s')) \quad (1)$$

$$\geq \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_\pi(s')) \quad (2)$$

$$\geq \sum_{s'} T(s'|s, \pi(s)) (R(s, \pi(s), s') + \gamma V_\pi(s')) \quad (3)$$

$$= V_\pi(s) \quad (4)$$

- Given the value of a policy  $V_\pi$ , can feed it through the optimal Bellman equation (2) to get a better policy  $\pi'$ . Denote  $V_\pi \xrightarrow{I} \pi'$ .

## Policy Improvement

$$\pi'(s) := \arg \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_\pi(s'))$$

**Exercise:** Prove  $\pi' \geq \pi$ .

# Policy Iteration

## Policy Improvement (I)

$$\pi'(s) \leftarrow \arg \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

## Policy Evaluation (E)

Solve

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

for  $V_{\pi}(s_1), V_{\pi}(s_2), \dots$

# Policy Iteration

## Policy Improvement (I)

$$\pi'(s) \leftarrow \arg \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

## Policy Evaluation (E)

Solve

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

for  $V_{\pi}(s_1), V_{\pi}(s_2), \dots$

- Start with arbitrary policy  $\pi_0$ .
- Note that  $\pi_*$  is fixed point of policy improvement.
- Alternate until policy is stable

$$\pi_0 \xrightarrow{E} V_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} V_{\pi_1} \xrightarrow{E} \pi_2 \xrightarrow{I} V_{\pi_2} \xrightarrow{E} \dots \xrightarrow{I} \pi_* \xrightarrow{E} V_{\pi_*} \xrightarrow{I} \pi_*$$

# Policy Iteration

## Policy Improvement (I)

$$\pi'(s) \leftarrow \arg \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

## Policy Evaluation (E)

Solve

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

for  $V_{\pi}(s_1), V_{\pi}(s_2), \dots$

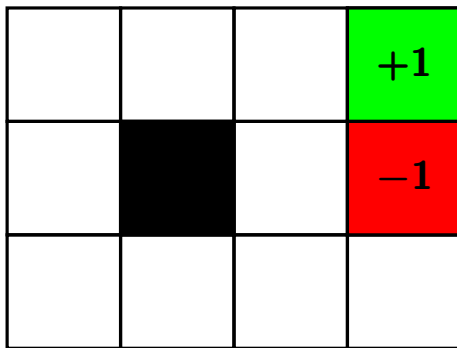
- Start with arbitrary policy  $\pi_0$ .
- Note that  $\pi_*$  is fixed point of policy improvement.
- Alternate until policy is stable

$$\pi_0 \xrightarrow{E} V_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} V_{\pi_1} \xrightarrow{E} \pi_2 \xrightarrow{I} V_{\pi_2} \xrightarrow{E} \dots \xrightarrow{I} \pi_* \xrightarrow{E} V_{\pi_*} \xrightarrow{I} \pi_*$$

**Theorem:** Policy iteration converges to optimal policy in finitely many steps!

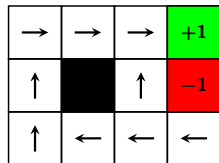


# Slippery Gridworld

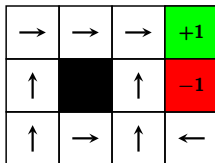


**Figure:** The slippery gridworld environment. The agent can move in any of the four cardinal directions. Receives reward (punishment?)  $\epsilon < 0$  on each timestep. Movement is noisy, with a 70% chance of moving in the desired direction, and 10% of moving in the other three directions. Episode ends when the agent lands in **heaven** +1 (and gets reward +1) or in **hell** -1 (and gets reward -1). The wall ■ is impassible, any movement that would walk into the wall, or off the grid, has no effect.

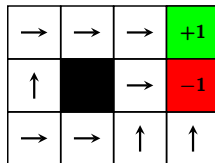
# Optimal Policy for Slippery Gridworld



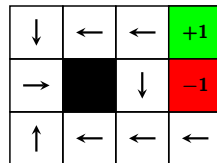
(a)  $\varepsilon = -0.01$



(b)  $\varepsilon = -0.1$



(c)  $\varepsilon = -2$



(d)  $\varepsilon = 0.01$

# Optimal Policy for Slippery Gridworld

|   |   |   |    |
|---|---|---|----|
| → | → | → | +1 |
| ↑ |   | ↑ | -1 |
| ↑ | ← | ← | ←  |

(a)  $\varepsilon = -0.01$

|   |   |   |    |
|---|---|---|----|
| → | → | → | +1 |
| ↑ |   | ↑ | -1 |
| ↑ | → | ↑ | ←  |

(b)  $\varepsilon = -0.1$

|   |   |   |    |
|---|---|---|----|
| → | → | → | +1 |
| ↑ |   | → | -1 |
| → | → | ↑ | ↑  |

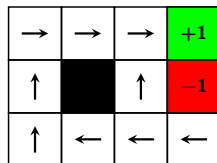
(c)  $\varepsilon = -2$

|   |   |   |    |
|---|---|---|----|
| ↓ | ← | ← | +1 |
| → |   | ↓ | -1 |
| ↑ | ← | ← | ←  |

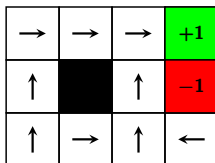
(d)  $\varepsilon = 0.01$

- ❷ **Patient:** The penalty is slight, so the agent takes the long way around to get to heaven.

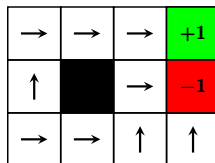
# Optimal Policy for Slippery Gridworld



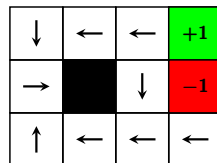
(a)  $\varepsilon = -0.01$



(b)  $\varepsilon = -0.1$



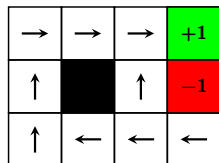
(c)  $\varepsilon = -2$



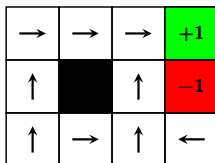
(d)  $\varepsilon = 0.01$

- ❶ **Patient:** The penalty is slight, so the agent takes the long way around to get to heaven.
- ❷ **Risky:** The penalty is moderate, so the agent takes the shortest path to heaven, risking hell along the way.

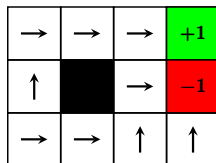
# Optimal Policy for Slippery Gridworld



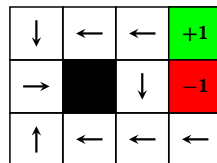
(a)  $\epsilon = -0.01$



(b)  $\epsilon = -0.1$



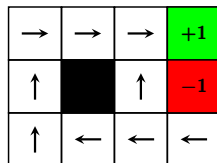
(c)  $\epsilon = -2$



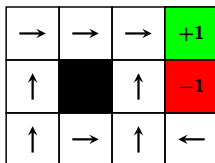
(d)  $\epsilon = 0.01$

- Ⓐ **Patient:** The penalty is slight, so the agent takes the long way around to get to heaven.
- Ⓑ **Risky:** The penalty is moderate, so the agent takes the shortest path to heaven, risking hell along the way.
- Ⓒ **Agony:** The penalty is severe, so the agent wants to end the episode as quickly as possible.

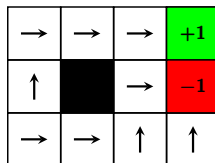
# Optimal Policy for Slippery Gridworld



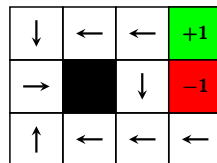
(a)  $\epsilon = -0.01$



(b)  $\epsilon = -0.1$



(c)  $\epsilon = -2$



(d)  $\epsilon = 0.01$

- (a) **Patient:** The penalty is slight, so the agent takes the long way around to get to heaven.
- (b) **Risky:** The penalty is moderate, so the agent takes the shortest path to heaven, risking hell along the way.
- (c) **Agony:** The penalty is severe, so the agent wants to end the episode as quickly as possible.
- (d) **Bliss:** The reward is positive on each timestep, so the agent runs away from heaven or hell to drag out the episode as long as possible.

- **Transition** distribution  $T : \mathcal{S} \times \mathcal{A} \rightarrow \Delta\mathcal{S}$ , sample  $s' \sim T(\cdot|s, a)$ .
- **Reward** function  $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}$ , write **reward**  $r_{t+1} := R(s_t, a_t, s_{t+1})$
- **Policy**  $\pi : \mathcal{S} \rightarrow \Delta\mathcal{A}$  (stochastic) or  $\pi : \mathcal{S} \rightarrow \mathcal{A}$  (deterministic), take  $a_t = \pi(s_t)$  or sample  $a_t \sim \pi(\cdot|s_t)$ .
- **History/Trajectory**  $s_0, a_0, r_1, s_1, a_1, r_2, s_2, a_2, r_3, s_3, \dots$
- **Agent** is the algorithm that selects the *policy* based on the *history*.
- **Discount**  $\gamma \in [0, 1)$ . Adjusts how far-sighted agent is.  $\gamma = 0$  is *myopic*.  $\gamma \rightarrow 1$  is *far-sighted*.
- **Return**  $G_t = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \dots = r_{t+1} + \gamma G_{t+1}$  is sum of *future discounted rewards*.
- **Goal** is to maximise  $G_t$ .
- **Value** function  $V_\pi(s) = \mathbb{E}_\pi[G_t|s_t = s]$ .
- **Bellman** equation  $V_\pi(s) = \sum_{s' \in \mathcal{S}} T(s'|s, a) (R(s, a, s') + \gamma V_\pi(s'))$
- **Policy Improvement**  $\pi'(s) \leftarrow \arg \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} T(s'|s, a) (R(s, a, s') + \gamma V_\pi(s'))$
- **Policy Evaluation** Solve  $V_\pi(s) = \sum_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} T(s'|s, a) (r + \gamma V_\pi(s'))$  for  $V_\pi(s_1), V_\pi(s_2), \dots$

---

$\Delta X$  is the set of all probability vectors  $[p_1, \dots, p_n]$  over outcomes in  $X$ .

# Problems with Policy Iteration

## Policy Improvement (I)

$$\pi_{n+1}(s) \leftarrow \arg \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi_n}(s'))$$

## Policy Evaluation (E)

Solve

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

for  $V_{\pi}(s_1), V_{\pi}(s_2), \dots$

- Requires white-box access to the environmental distribution  $T$  and reward function  $R$ .
- Only works for environments with few enough states and actions to sweep through.



# Problems with Policy Iteration

## Policy Improvement (I)

$$\pi_{n+1}(s) \leftarrow \arg \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi_n}(s'))$$

## Policy Evaluation (E)

Solve

$$V_{\pi}(s) = \sum_a \pi(a|s) \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

for  $V_{\pi}(s_1), V_{\pi}(s_2), \dots$

- Requires white-box access to the environmental distribution  $T$  and reward function  $R$ .
- Only works for environments with few enough states and actions to sweep through.

For the moment, we weaken only the first assumption. Environment is black box, sample percepts  $(s', r)$  given state-action pair  $(s, a)$  from unknown  $T$  and  $R$ .

# Temporal Difference Learning

- **Goal:** Perform policy evaluation without access to environmental distribution.

# Temporal Difference Learning

- **Goal:** Perform policy evaluation without access to environmental distribution.
- **Motivation:** Consider once again the value function:

$$V_{\pi}(s) = \mathbb{E}_{\substack{a=\pi(s) \\ s' \sim T(\cdot|s,a)}} [R(s, a, s') + \gamma V_{\pi}(s')]$$

# Temporal Difference Learning

- **Goal:** Perform policy evaluation without access to environmental distribution.
- **Motivation:** Consider once again the value function:

$$V_{\pi}(s) = \mathbb{E}_{\substack{a=\pi(s) \\ s' \sim T(\cdot|s,a)}} [R(s, a, s') + \gamma V_{\pi}(s')]$$

On timestep  $t$ , this is *on average*, equal to the actual reward  $r_{t+1}$ , plus the discounted value of the actual next state  $s_{t+1}$ .

$$V_{\pi}(s_t) \approx r_{t+1} + \gamma V_{\pi}(s_{t+1})$$

# Temporal Difference Learning

- **Goal:** Perform policy evaluation without access to environmental distribution.
- **Motivation:** Consider once again the value function:

$$V_{\pi}(s) = \mathbb{E}_{\substack{a=\pi(s) \\ s' \sim T(\cdot|s,a)}} [R(s, a, s') + \gamma V_{\pi}(s')]$$

On timestep  $t$ , this is *on average*, equal to the actual reward  $r_{t+1}$ , plus the discounted value of the actual next state  $s_{t+1}$ .

$$V_{\pi}(s_t) \approx r_{t+1} + \gamma V_{\pi}(s_{t+1})$$

We define the **TD-Error** as the difference

$$\delta_t := r_{t+1} + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)$$

# Temporal Difference Learning

- **Goal:** Perform policy evaluation without access to environmental distribution.
- **Motivation:** Consider once again the value function:

$$V_{\pi}(s) = \mathbb{E}_{\substack{a=\pi(s) \\ s' \sim T(\cdot|s,a)}} [R(s, a, s') + \gamma V_{\pi}(s')]$$

On timestep  $t$ , this is *on average*, equal to the actual reward  $r_{t+1}$ , plus the discounted value of the actual next state  $s_{t+1}$ .

$$V_{\pi}(s_t) \approx r_{t+1} + \gamma V_{\pi}(s_{t+1})$$

We define the **TD-Error** as the difference

$$\delta_t := r_{t+1} + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)$$

This then gives us an update rule to improve on our estimate  $\hat{V}_{\pi}$  of  $V_{\pi}$ , similar to SGD, called  $TD(0)$ .

$$\hat{V}_{\pi}(s_t) \leftarrow \hat{V}_{\pi}(s_t) + \alpha \delta_t \equiv \hat{V}_{\pi}(s_t) + \alpha (r_{t+1} + \gamma \hat{V}_{\pi}(s_{t+1}) - \hat{V}_{\pi}(s_t))$$

where  $\alpha \in (0, 1]$  is the **learning rate**.

**Theorem:** Given  $\alpha \rightarrow 0$  in a nice way,  $TD(0)$  guaranteed to converge to  $V_{\pi}$ .

# Q-Value

- **Q-value** is the expected return from state  $s$ , taking action  $a$ , and thereafter following policy  $\pi$ .

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | s_t = s, a_t = a]$$

# Q-Value

- **Q-value** is the expected return from state  $s$ , taking action  $a$ , and thereafter following policy  $\pi$ .

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | s_t = s, a_t = a]$$

Contrast with the value function

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | s_t = s]$$



# Q-Value

- **Q-value** is the expected return from state  $s$ , taking action  $a$ , and thereafter following policy  $\pi$ .

$$Q_{\pi}(s, a) = \mathbb{E}_{\pi}[G_t | s_t = s, a_t = a]$$

Contrast with the value function

$$V_{\pi}(s) = \mathbb{E}_{\pi}[G_t | s_t = s]$$

## Q-value Bellman

$$Q_{\pi}(s, a) = \sum_{s'} T(s' | s, a) (R(s, a, s') + \gamma Q_{\pi}(s', a'))$$

where  $a' = \pi(s')$

## Optimal Q-value Bellman

$$Q_{*}(s, a) = \sum_{s'} T(s' | s, a) \left( R(s, a, s') + \max_{a'} Q_{*}(s', a') \right)$$

# Q-value vs. Value

**Exercise:** Can state  $Q$  in terms of  $V$ , and vice-versa.

$$Q_{\pi}(s, a) = \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_{\pi}(s'))$$

$$V_{\pi}(s) = \sum_{s'} T(s'|s, \pi(s)) (R(s, a, s') + \gamma Q_{\pi}(s', \pi(s')))$$

$$Q_{*}(s, a) = \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_{*}(s'))$$

$$V_{*}(s) = \max_a \sum_{s'} T(s'|s, a) \left( R(s, a, s') + \gamma \max_{a'} Q_{*}(s', a') \right)$$

# Motivation for Q-Value

- So far, we have been learning a policy  $\pi$ , and using  $\pi$  to compute  $V_\pi$ .

# Motivation for Q-Value

- So far, we have been learning a policy  $\pi$ , and using  $\pi$  to compute  $V_\pi$ .
- Even if we were given  $V_*$  directly, can't recover  $\pi_*$  without white-box access to  $T$  and  $R$  (environment).

$$\pi_*(s) = \arg \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_*(s'))$$

# Motivation for Q-Value

- So far, we have been learning a policy  $\pi$ , and using  $\pi$  to compute  $V_\pi$ .
- Even if we were given  $V_*$  directly, can't recover  $\pi_*$  without white-box access to  $T$  and  $R$  (environment).

$$\pi_*(s) = \arg \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_*(s'))$$

- However, given  $Q_*$ , we can directly recover  $\pi_*$

$$\pi_*(s) = \arg \max_a Q_*(s, a)$$

# Motivation for Q-Value

- So far, we have been learning a policy  $\pi$ , and using  $\pi$  to compute  $V_\pi$ .
- Even if we were given  $V_*$  directly, can't recover  $\pi_*$  without white-box access to  $T$  and  $R$  (environment).

$$\pi_*(s) = \arg \max_a \sum_{s'} T(s'|s, a) (R(s, a, s') + \gamma V_*(s'))$$

- However, given  $Q_*$ , we can directly recover  $\pi_*$

$$\pi_*(s) = \arg \max_a Q_*(s, a)$$

- **Idea:** Learn  $Q_*$  instead, argmax to recover policy  $\pi_*$

# SARSA: On-Policy TD Control

Apply same argument as TD(0) to the Q-Value

$$Q_*(s, a) = \mathbb{E}_{s' \sim T(\cdot|s,a)} [R(s, a, s') + \gamma Q_*(s', \pi_*(s'))]$$

# SARSA: On-Policy TD Control

Apply same argument as TD(0) to the Q-Value

$$Q_*(s, a) = \mathbb{E}_{s' \sim T(\cdot | s, a)} [R(s, a, s') + \gamma Q_*(s', \pi_*(s'))]$$

On timestep  $t$ , this is “*on average*”, equal to the actual reward  $r_{t+1}$ , plus the discounted Q-value of the actual next state-action pair  $s_{t+1}, a_{t+1}$ .

$$Q_*(s_t, a_t) \approx r_{t+1} + \gamma Q_*(s_{t+1}, a_{t+1})$$

Update depends on  $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$ .



# SARSA: On-Policy TD Control

Apply same argument as TD(0) to the Q-Value

$$Q_*(s, a) = \mathbb{E}_{s' \sim T(\cdot|s,a)} [R(s, a, s') + \gamma Q_*(s', \pi_*(s'))]$$

On timestep  $t$ , this is “on average”, equal to the actual reward  $r_{t+1}$ , plus the discounted Q-value of the actual next state-action pair  $s_{t+1}, a_{t+1}$ .

$$Q_*(s_t, a_t) \approx r_{t+1} + \gamma Q_*(s_{t+1}, a_{t+1})$$

Update depends on  $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$ .

## SARSA Update Rule

$$\hat{Q}_*(s_t, a_t) \leftarrow \hat{Q}_*(s_t, a_t) + \alpha (r_{t+1} + \gamma \hat{Q}_*(s_{t+1}, a_{t+1}) - \hat{Q}_*(s_t, a_t))$$

where  $\alpha \in (0, 1]$  is the **learning rate**.

# SARSA: On-Policy TD Control

Apply same argument as TD(0) to the Q-Value

$$Q_*(s, a) = \mathbb{E}_{s' \sim T(\cdot|s,a)} [R(s, a, s') + \gamma Q_*(s', \pi_*(s'))]$$

On timestep  $t$ , this is “on average”, equal to the actual reward  $r_{t+1}$ , plus the discounted Q-value of the actual next state-action pair  $s_{t+1}, a_{t+1}$ .

$$Q_*(s_t, a_t) \approx r_{t+1} + \gamma Q_*(s_{t+1}, a_{t+1})$$

Update depends on  $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$ .

## SARSA Update Rule

$$\hat{Q}_*(s_t, a_t) \leftarrow \hat{Q}_*(s_t, a_t) + \alpha (r_{t+1} + \gamma \hat{Q}_*(s_{t+1}, a_{t+1}) - \hat{Q}_*(s_t, a_t))$$

where  $\alpha \in (0, 1]$  is the **learning rate**.

Actions drawn from  $\varepsilon$ -greedy strategy

$$\pi^{\varepsilon\text{-greedy}}(s) = \begin{cases} \text{do random action} & \text{prob } \varepsilon \\ \arg \max_a \hat{Q}_*(s, a) & \text{prob } 1 - \varepsilon \end{cases}$$

# SARSA: On-Policy TD Control

Apply same argument as TD(0) to the Q-Value

$$Q_*(s, a) = \mathbb{E}_{s' \sim T(\cdot | s, a)} [R(s, a, s') + \gamma Q_*(s', \pi_*(s'))]$$

On timestep  $t$ , this is “on average”, equal to the actual reward  $r_{t+1}$ , plus the discounted Q-value of the actual next state-action pair  $s_{t+1}, a_{t+1}$ .

$$Q_*(s_t, a_t) \approx r_{t+1} + \gamma Q_*(s_{t+1}, a_{t+1})$$

Update depends on  $(s_t, a_t, r_t, s_{t+1}, a_{t+1})$ .

## SARSA Update Rule

$$\hat{Q}_*(s_t, a_t) \leftarrow \hat{Q}_*(s_t, a_t) + \alpha (r_{t+1} + \gamma \hat{Q}_*(s_{t+1}, a_{t+1}) - \hat{Q}_*(s_t, a_t))$$

where  $\alpha \in (0, 1]$  is the **learning rate**.

Actions drawn from  $\varepsilon$ -greedy strategy

$$\pi^{\varepsilon\text{-greedy}}(s) = \begin{cases} \text{do random action} & \text{prob } \varepsilon \\ \arg \max_a \hat{Q}_*(s, a) & \text{prob } 1 - \varepsilon \end{cases}$$

**Theorem:** Under “niceness” conditions SARSA guaranteed to converge to  $Q_*$ . 

# Q-Learning: Off-Policy TD Control

- Why learn from  $a_{t+1}$  when it was a random exploration action? Why not instead learn from the best<sup>1</sup> action  $\arg \max_{a'} Q(s_{t+1}, a')$  that should have been taken?

---

<sup>1</sup>according to the current Q-value estimates

# Q-Learning: Off-Policy TD Control

- Why learn from  $a_{t+1}$  when it was a random exploration action? Why not instead learn from the best<sup>1</sup> action  $\arg \max_{a'} Q(s_{t+1}, a')$  that should have been taken?

## Q-Learning Update Rule

$$\hat{Q}_*(s_t, a_t) \leftarrow \hat{Q}_*(s_t, a_t) + \alpha \left( r_{t+1} + \gamma \max_{a'} \hat{Q}(s_{t+1}, a') - \hat{Q}(s_t, a_t) \right)$$

Actions taken via  $\varepsilon$ -greedy strategy over  $\hat{Q}_*(s, a)$ .

---

<sup>1</sup>according to the current Q-value estimates

# Q-Learning: Off-Policy TD Control

- Why learn from  $a_{t+1}$  when it was a random exploration action? Why not instead learn from the best<sup>1</sup> action  $\arg \max_{a'} Q(s_{t+1}, a')$  that should have been taken?

## Q-Learning Update Rule

$$\hat{Q}_*(s_t, a_t) \leftarrow \hat{Q}_*(s_t, a_t) + \alpha \left( r_{t+1} + \gamma \max_{a'} \hat{Q}(s_{t+1}, a') - \hat{Q}(s_t, a_t) \right)$$

Actions taken via  $\varepsilon$ -greedy strategy over  $\hat{Q}_*(s, a)$ .

**Theorem:** Under “niceness” conditions Q-learning guaranteed to converge to  $Q_*$ .

---

<sup>1</sup>according to the current Q-value estimates

# SARSA v.s. Q-Learning

## SARSA Update Rule

$$\hat{Q}_*(s_t, a_t) \leftarrow \hat{Q}_*(s_t, a_t) + \alpha \left( r_{t+1} + \gamma \hat{Q}_*(s_{t+1}, a_{t+1}) - \hat{Q}_*(s_t, a_t) \right)$$

## Q-Learning Update Rule

$$\hat{Q}_*(s_t, a_t) \leftarrow \hat{Q}_*(s_t, a_t) + \alpha \left( r_{t+1} + \gamma \max_{a'} \hat{Q}(s_{t+1}, a') - \hat{Q}(s_t, a_t) \right)$$

- Q-Learning (usually) tends to converge faster than SARSA, and chooses more aggressive/risky moves
- SARSA learns from the moves that were actually taken, including any exploration
- In “risky” environments, SARSA will learn to avoid getting near dangerous situations (to avoid accidentally taking a very bad exploratory move). Q-Learning will not.